13 Increase the score

First the score is increased by the value of the `points` parameter. Next the new speed and interval of the eggs are calculated by multiplying their values by the difficulty factor. Finally, the text on the screen is updated with the new score. Add this new function beneath `check_catch()`.


```
    root.after(100, check_catch)

def increase_score(points):
    global score, egg_speed, egg_interval
    score += points
    egg_speed = int(egg_speed * difficulty_factor)
    egg_interval = int(egg_interval * difficulty_factor)
    c.itemconfigure(score_text, text='Score: ' + str(score))
```

Add to the player's score.

This line updates the text that shows the score.

Catch those eggs!

Now that you've got all the shapes and functions needed for the game, all that's left to add are the controls for the egg catcher and the commands that start the game.

14 Set up the controls

The `move_left()` and `move_right()` functions use the coordinates of the catcher to make sure it isn't about to leave the screen. If there's still space to move to, the catcher shifts horizontally by 20 pixels. These two functions are linked to the left and right arrow keys on the keyboard using the `bind()` function. The `focus_set()` function allows the program to detect the key presses. Add the new functions beneath the `increase_score()` function.

```
    c.itemconfigure(score_text, text='Score: \
                ' + str(score))

def move_left(event):
    (x1, y1, x2, y2) = c.coords(catcher)
    if x1 > 0:
        c.move(catcher, -20, 0)

def move_right(event):
    (x1, y1, x2, y2) = c.coords(catcher)
    if x2 < canvas_width:
        c.move(catcher, 20, 0)

c.bind('<Left>', move_left)
c.bind('<Right>', move_right)
c.focus_set()
```

Has the catcher reached the left-hand wall?

If not, move the catcher left.

Has the catcher reached the right-hand wall?

If not, move the catcher right.

These lines call the functions when the keys are pressed.